COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	08/18/2026	**DATE SUBMITTED:**	08/–/26
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 6 [ACTIVITY NAME] Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 6-1. Data Generated and Observations

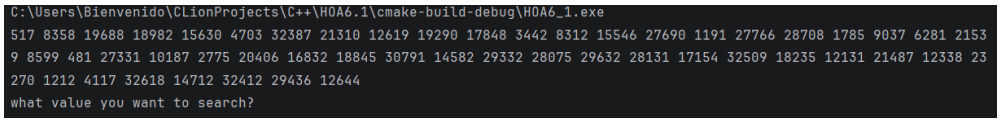
Screenshot	
Observation	I thought from the start it will display same numbers from the previous but when you run it multiple times the numbers are generated randomly from what i've seen and another thing is its digit is 1-5.

Table 6-2a. Linear Search for Arrays

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingalgo.h"

const int max_size = 50;

int main() {
    //generate random values:
    int dataset[max_size];
    srand(time(0));
    for(int i=0; i<max_size; i++){
        dataset[i] = rand();
    }
    for(int i=0; i < max_size; i++){
        std::cout <<dataset[i] << " ";
    }
    std::cout<<std::endl;
    //create a user input to ask what value to find:
    int toFind;
    std::cout<< "what value you want to search? ";
    std::cin >> toFind;

    linearSearchArr(dataset, max_size, toFind);
```

	``
	    return 0;
	
	}
	``

Output	``` C:\Users\Bienvenido\CLionProjects\C++\HOA6.1\cmake-build-debug\HOA6_1.exe 3133 32615 9273 26112 20068 4286 2324 3090 131 9887 30447 5955 14063 30881 10588 28138 13171 8944 26399 10697 7828 2698 9619 15845 26137 20911 31562 11926 25803 28818 9716 4058 14202 18005 10114 15854 25408 13876 16011 1056 12554 9473 15854 9695 25171 27344 1977 32276 9011 3315 what value you want to search? 9011 ```
Observation	When the program runs, it generate a 1 to 5 digit number and up to 50 random numbers and then when you input the number that came from the random generated numbers, it uses linearSearchArr() to run it in order from 0 up to 49 until it found its target, once it found the number its needing, it will say the " the search is success" and the code will stop.

Table 6-2b. Linear Search for Linked List

Code	```
	#include <iostream>
	#include <cstdlib>
	#include <time.h>
	#include "nodes.h"
	#include "searchingalgo.h"
	
	const int max_size = 50;
	
	int main() {
	    /*//generate random values:
	    int dataset[max_size];
	    srand(time(0));
	    for(int i=0; i<max_size; i++){
	        dataset[i] = rand();
	    }
	    for(int i=0; i < max_size; i++){
	        std::cout <<dataset[i] << " ";
	    }
	    std::cout<<std::endl;
	    //create a user input to ask what value to find:
	    int toFind;
	    std::cout<< "what value you want to search? ";
	    std::cin >> toFind;
	
	    linearSearchArr(dataset, max_size, toFind);
	    */
	
	    //linear search:
	
	    //linked list:
	    Node<char>* name1 = new_node('B');
	    Node<char>* name2 = new_node('I');
	``

```
    Node<char>* name3 = new_node('E');
    Node<char>* name4 = new_node('N');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;

    std::cout<<"Linear Search for Linked List result: ";
    linearSearchLL(name1, 'N');


}
```

Output

```
C:\Users\Bienvenido\CLionProjects\C++\HOA6.1\cmake-build-debug\HOA6_1.exe
Linear Search for Linked List result: the search is success
```

Observation

in this one, the implementation, this one is a singly linked list then it is constructed using the nodes and when the linearsearchll is called, it tranverse through the list of node, node by node which starts on the head pointer, with the value is declared in the main driver which finding the letter N, the program runs through every single node until it met the requirements, once it reaches the value it need to find the condition changes to true, in result of the output was "the search is success", if the value was not present in the node, it will loop until it reaches the nullptr and the result will be the element is not found.

Table 6-3a. Binary Search for Arrays

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <algorithm>
#include "nodes.h"
#include "searchingalgo.h"

const int max_size = 50;

int main() {
    int dataset[max_size];
    srand(time(0));

    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
    }

    // Binary search REQUIRES a sorted array
    std::sort(dataset, dataset + max_size);

    std::cout << "Sorted Dataset: ";
    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
```

```
    }
    std::cout << std::endl;

    int toFind;
    std::cout << "What value do you want to search? ";
    std::cin >> toFind;

    binarySearchArr(dataset, max_size, toFind);

    return 0;
}
```

Header

```
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
 int low = 0;
 int up = arrSize - 1;

 while (low <= up) {
   int mid = (low + up) / 2;

   if (dataFind == data[mid]) {
     std::cout << "Search element is found!\n";
     return;
   }
   else if (dataFind < data[mid]) {
     up = mid - 1;
   }
   else {
     low = mid + 1;
   }
 }

 std::cout << "Search element is not found\n";
}
```

Output

```
C:\Users\Bienvenido\CLionProjects\C++\HOA6.1\cmake-build-debug\HOA6_1.exe
Sorted Dataset: 295 339 2128 2411 2540 2573 3557 3970 5318 6037 6475 6549 6869 6951 7094 8012 8215 8378 11179 11720 1178
0 11949 13095 14106 14886 15421 16774 17223 17238 17411 17944 18263 18867 19098 19931 20039 20444 20894 21051 21280 2167
7 21874 23308 25057 25801 26363 26480 26721 28475 30221
What value do you want to search? 30221

Search element is found!
```

Observation

In this part of activity this program also generated an 50 random numbers and from the output you can see it is also assorted in ascending order using the sort library, once the user puts the value for example i use the value 30221, the binarysearchArr search the range to compare it witht he target value until it found its target value, displaying "search element is found"

Table 6-3b. Binary Search for Linked List

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <algorithm>
#include "nodes.h"
#include "searchingalgo.h"

const int max_size = 50;

int main() {
     /*//generate random values:
       int dataset[max_size];
       srand(time(0));
       for(int i=0; i<max_size; i++){
           dataset[i] = rand();
       }
       for(int i=0; i < max_size; i++){
           std::cout <<dataset[i] << " ";
       }
       std::cout<<std::endl;
       //create a user input to ask what value to find:
       int toFind;
       std::cout<< "what value you want to search? ";
       std::cin >> toFind;

       linearSearchArr(dataset, max_size, toFind);*/

       //linked list:
       Node<char>* name1 = new_node('B');
       Node<char>* name2 = new_node('I');
       Node<char>* name3 = new_node('E');
       Node<char>* name4 = new_node('N');

       name1->next = name2;
       name2->next = name3;
       name3->next = name4;

       std::cout<<"Binary Search for Linked List result: ";
       binarySearchLL(name1, 'N');

   /*int dataset[max_size];
  srand(time(0));

  for (int i = 0; i < max_size; i++) {
      dataset[i] = rand();
  }

  // Binary search REQUIRES a sorted array
  std::sort(dataset, dataset + max_size);

  std::cout << "Sorted Dataset: ";
  for (int i = 0; i < max_size; i++) {
      std::cout << dataset[i] << " ";
```

```
    }
    std::cout << std::endl;

    int toFind;
    std::cout << "What value do you want to search? ";
    std::cin >> toFind;

    binarySearchArr(dataset, max_size, toFind);*/

     return 0;

}
```

Output

```
C:\Users\Bienvenido\CLionProjects\C++\HOA6.1\cmake-build-debug\HOA6_1.exe
Binary Search for Linked List result: Found: N

Process finished with exit code 0
```

Observation

the function binarySeachLL locates the node that contains the letter N by doing start in the middle node of the linked list using the slow and fast pointer, since the target is letter N and is alphabetically greater than the middle element letter E, after that the algorithm updates the starting boundary to evaluate the remaining sublist, successfully finding a match and outputting

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.
Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Code

```
int main() {
    int nums[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int len = 10;
    int k = 18;

    Node<int>* head = new_node(nums[0]);
    Node<int>* ptr = head;
    for (int i = 1; i < len; i++) {
        ptr->next = new_node(nums[i]);
        ptr = ptr->next;
    }

    // Problem 1
    std::cout << "[Array Search]\n";
    countComparisonsArr(nums, len, k);
    std::cout << "\n[Linked List Search]\n";
```

```
countComparisonsLL(head, k);
```

Output

```
C:\Users\Bienvenido\CLionProjects\
[Array Search]
the search is success
Comparisons needed: 2

[Linked List Search]
the search is success
Comparisons needed: 2
```

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

```
//Problem 2
std::cout << "\nProblem 2\n";
countOccurrencesArr(nums, len, k);
countOccurrencesLL(head, k);
```

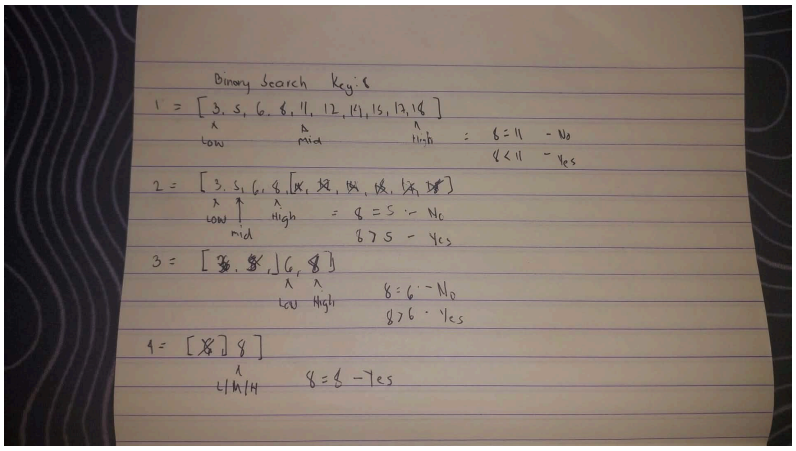
```
// Function to count repeating instances of key 'k' (Problem 2)
template <typename T>
void countOccurrencesArr(T data[], int arrSize, T k) {
 int count = 0;
 for (int i = 0; i < arrSize; i++) {
   if (data[i] == k) {
     count++;
   }
 }
 std::cout << "Repeating instances of " << k << " in Array: "
<< count << "\n";
}

template <typename T>
void countOccurrencesLL(Node<T>* node, T k) {
 int count = 0;
 while (node != nullptr) {
   if (node->data == k) {
     count++;
   }
   node = node->next;
 }
 std::cout << "Repeating instances of " << k << " in Linked
List: " << count << "\n";
}
```

Output	Problem 2 Repeating instances of 18 in Array: 2 Repeating instances of 18 in Linked List: 2

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Diagram



Code

```
int list[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int size = sizeof(list) / sizeof(list[0]);
int key = 8;

//Problem 3
std::cout << "\nProblem 3\n";
binarySearchArr(list, size, key);
```

```
//since the function for binary search is already here
//we can say that we can also use this for problem 3
//problem 3
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
 int low = 0;
 int up = arrSize - 1;

 while (low <= up) {
   int mid = (low + up) / 2;

   if (dataFind == data[mid]) {
     std::cout << "Search element is found!\n";
     return;
   }
```

```
    else if (dataFind < data[mid]) {
      up = mid - 1;
    }
    else {
      low = mid + 1;
    }
  }

  std::cout << "Search element is not found\n";
}
```

Output

```
Problem 3
Search element is found!
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code

```
//Since both problem 3 and 4 are the same, one will modify
int list[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int size = sizeof(list) / sizeof(list[0]);
int key = 8;

//Problem 3
std::cout << "\nProblem 3\n";
binarySearchArr(list, size, key);

//Problem 4
std::cout << "\nProblem 4\n";
std::cout << "searching for binary number " << key << " in the
array\n";
binaryRecursive(list, 0, size - 1, key);
```

```
//function for problem 4
template <typename T>
void binaryRecursive(T data[], int low, int up, T dataFind) {
 if (low > up) {
   std::cout << "search failure\n";
   return;
 }

 int mid = (low + up) / 2;

 if (data[mid] == dataFind) {
   std::cout << "search success\n";
   return;
 }
 else if (dataFind < data[mid]) {
   binaryRecursive(data, low, mid - 1, dataFind);
 }
```

```
else {
  binaryRecursive(data, mid + 1, up, dataFind);
 }
}
```

Output

```
Problem 4
searching for binary number 8 in the array
search success
```

Conclusion:

Summary of lessons learned

- In this whole activity, i found out that when it comes to linear search, it checks the element one by one until it found its value target but it works on both sorted and unsorted but it can be slow for large datasets. Binary search is much faster because it repeatedly cuts the search area in half, though the list must be sorted first. Arrays allow you to jump straight to any index, which binary search needs to work efficiently.

Analysis of the procedure

- I found out that it took 2 steps for both array adn inked list as the 18 was near, Counting all copies of 18 needed to check every single item from start to finish, but was able to find two total copies in both data structures. Then, iteratively using a binary search, it quickly found the target number 8 in four steps by halving the ordered array repeatedly. Recursive binary search performed the same four steps as Problem 3, except it used self-referencing function calls rather than a normal loop.

Analysis of the supplementary activity

- Working throughout the whole activity these exercises also gave me a much better understanding of how algorithm choices directly impact of your code efficiency and program performance.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- Well it took me a whole week to finish the code, i did good in this part since once i figured out what i can do in this part of each activity, its easy to get, although it get me a lot of confusion but i did well adn another isI dealt with a lot of initial confusion while trying to understand how each algorithm functioned. However, once I figured out what needed to be done for each part, the implementation became much easier to complete and follow through.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.